Group Moderation Under End-to-End Encryption

Sarah Scheffler
Massachusetts Institute of Technology
USA
sscheff@csail.mit.edu

Jonathan Mayer
Princeton University
USA
jonathan.mayer@princeton.edu

ACM Reference Format:
Sarah Scheffler and Jonathan Mayer. 2024. Group Moderation Under End-to-End Encryption. In . ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Online services are increasingly adopting end-to-end encryption (E2EE), a valuable safeguard for user data. In an E2EE design, only parties to a communication have access to plaintext—not the service provider—which greatly reduces security and privacy risks. E2EE is used extensively in messaging (e.g., Apple's iMessage [2021a], Meta's WhatsApp [Kent 2018], Google Messages [2022], and Signal [2013]), group conversations (e.g., Matrix [2019] and Wickr [Amazon Web Services 2023]), and data storage (e.g., Apple's Advanced Data Protection for iCloud [2022b]).

The rise of E2EE poses challenges for content and community moderation, which—as presently implemented—typically involves a trusted online service that can operate on user content and communications state. These approaches to moderation are fundamentally incompatible with the privacy guarantees of E2EE. Governments worldwide have sounded alarm about how E2EE might enable the distribution of harmful content, such as child abuse material. Some have called for a moratorium on further E2EE deployment [US Department of Justice 2020], and pending legislation in a number of jurisdictions could require E2EE services to implement forms of content moderation where technically feasible [European Union 2022; United Kingdom 2021; US Senate 2023].

In recent work, we systematized the landscape of goals, challenges, and prior protocols for content moderation under E2EE [Scheffler and Mayer 2023]. Our project centered on the one-on-one messaging and data storage use cases, because those applications have attracted the greatest scrutiny from policymakers. We expressly left group moderation, a related but distinct problem area, out of scope. This project complements our prior work, addressing the group moderation gap.

In the group moderation setting, users often govern themselves, typically by designating one or more administrators or moderators within the group. These users possess heightened permissions to manage allowed content, group membership, and group metadata. Group moderation may occur at a small scale, such as in a small group chat, or at a larger scale, such as in a many-participant chat room, forum, social platform, or file sharing service.

We make three contributions in this extension of our prior work. First, we offer an abstraction of essential primitives for group moderation under E2EE: shared state management and key management (Section 3). We identify three main attacker goals: *Affect Honest User* (AHU), *Bypass Moderation Alone* (BMA), and *Malicious Subgroup* (MS), which we discuss further in Section 3.

Second, we identify common group moderation functionality in popular non-E2EE applications that allow interaction among a group, like Slack [2023b], Discord [2023a], Facebook Groups [2023], YouTube [2023], Internet Relay Chat [2000a], and an online forum [2023] (Section 4). We analyze the feasibility of implementing this functionality by building application-level content moderation on top of an existing E2EE messaging scheme, without modifying the cryptographic layer. We examine the risks from malicious users within a group, identifying how these users can or cannot evade application-level content moderation.

Our third contribution is a synthesis of existing methods for content moderation in E2EE, including the moderation options deployed in popular E2EE applications like Signal [2013], WhatsApp [Kent 2018], iMessage [2021a], and Element [2023a] (built on Matrix [2019]) in Section 5. We also explore uncommon and hypothetical group content moderation designs that incorporate cryptographic elements, including in anonymous networks, in Section 6.

Our main insight is that application-level content moderation built on top of an E2EE messaging scheme is, under some mild assumptions, sufficient to ensure that malicious group members' actions do not *Affect Honest Users*, a primary goal of much content moderation both in and out of encrypted settings. Moderation of E2EE communication by an existing group admin or moderator also has the benefit of leaving the privacy guarantee of E2EE unaffected. However, difficulties remain in both theory and practice for the other moderation goals of preventing malicious users from receiving un-moderated content or sending it to other malicious users; these settings are closer to the difficult one-to-one E2EE scenario. Additionally, we see in Section 5 that actual implementation of the feasible group moderation methods is highly variable among popular E2EE group chat applications.

2 WHY GROUP MODERATION?

With a growing number of spaces on the internet where communities can communicate with each other online, *moderation* has become a common requirement. In general, content moderation refers to the process of detecting and taking action on content or users that are problematic in some way (see [Goldman 2021; Kaye 2019]). This can be done for a variety of reasons and in a variety of contexts, including but not limited to: moving "off-topic" content from one forum thread to another, removing hate speech or slurs,

This work is licensed under a Creative Commons Attribution International 4.0 License.
CSLAW'24, March 12-13, 2024, Boston, MA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0333-1/24/03.
https://doi.org/10.1145/3614407.3643704.

blurring adult content, cautioning users about misinformation, reporting and blocking users who are harassing or spamming other users, removing illegal content, and much more.

There are a variety of ways to determine and communicate what content should be moderated. Sometimes official rules are stated in documentation, such as Community Guidelines or a Terms of Service document. Sometimes moderation is largely left up to human discretion. Moderation can also be conducted automatically by a centralized content platform (e.g., Facebook automatically detects and removes problematic content [2022]) or within a group (e.g., Discord's AutoMod [2023]). Some services or groups rely on a combination of human and automated processes.

Fusing encryption and content moderation is a notoriously difficult task. In prior works we described both the technical difficulty [Scheffler and Mayer 2023] and the policy challenges [Scheffler et al. 2023] posed by moderation systems that attempt to detect specific content sent between two malicious users.

But moderation can take many different forms, and has different goals. In groups, sometimes there is already a designated human moderator or administrator who is trusted to monitor the group's conversation and ensure the users have a pleasant experience. A moderator is generally empowered with some degree of social and technical control over group interactions. A moderator might set or enforce limits on what kinds of content are and aren't allowed in different settings, what users' own powers are to send, receive, or edit content, and enforce penalties when rule violations occur. The moderation process could be fully manual, or the human moderator may deploy automated tools to assist in the process.

For groups that already have a moderator who is trusted with additional powers over the group, this provides a reasonable candidate for performing moderation tasks in encrypted settings—the moderator is already trusted to honestly deploy the tools at their disposal, and already has access to plaintext. Moderation can even be performed without ever needing to report content outside the group itself. This is a much more approachable task than the more heavy-handed detection of illegal content in one-to-one E2EE communication. In the rest of this work, we explore the capabilities and limitations of this moderation model. In Section 4 will see that this form of moderation is well-suited to protect the experiences of honest users, but that it falls short when attempting to prevent malicious users from bypassing moderation in their own applications.

3 GROUP MODERATION MODEL

As we discussed in Section 2, there are numerous actions a particular moderator (automated or human) might want to perform, for example, give or revoke a user's permissions to post, send, or edit, blur or remove NSFW content, or file a report. These are all actions that a moderator might want to perform *regardless* of whether or not the group conversation is end-to-end encrypted.

For applications built on top of an E2EE messaging layer, which moderation tasks can be performed at the application layer, and which require some kind of interaction with the E2EE messaging layer? At a high level, we informally observe that adding a member to the group requires at least some action on the E2EE layer, since the new group member must receive keys to participate in the conversation. But what about other functionalities?

In this section, we define the model under which we will analyze the answer to this question, and define three possible attack settings. Looking ahead to Section 4, we will see that many common content moderation tasks can be accomplished only at the application layer, however, adversarial group members with flexible application code will still likely be able to bypass content moderation in some ways on their own devices without additional cryptographic enforcement.

We note that our model for adversarial users allows them to maliciously alter application code. The extent to which real-world adversarial users have this capability depends on the application itself. Some communications systems, like Matrix, are specifically designed so that communication happens over a minimally-restrictive (possibly encrypted) interface [Matrix 2023b]. End users may be part of different federated servers, and may interact with the system using many different clients, and so we make the assumption that users have a high amount of control of the application code. However, some applications make it much more difficult to participate in communication with altered application code, and in these settings real-world adversaries may be more limited.

3.1 E2EE Messaging and Application Layers

In our model we assume that N parties, called *users*, are each using an *application* that is built atop an *E2EE messaging layer* in which the N parties all form a group. An *honest* user runs the application as normal; a *malicious* user may change the way the application works, and can make arbitrary calls to the E2EE messaging layer. (More details on what it means to be a "group member" can be found in Appendix A.1.)

We refer to the underlying E2EE communication infrastructure as the *E2EE messaging layer* which provides at least confidentiality (informally, unauthorized users cannot read messages), authenticity (informally, malicious users and external entities cannot impersonate honest users), and known group membership (all users know the list of group members). One highly-relevant option for the E2EE messaging layer is Messaging Layer Security (MLS) [Barnes et al. 2023; Beurdouche et al. 2023], a standard for end-to-end encrypted communications released in March 2023. In this work, we adopt some of the MLS ideas and nomenclature, especially the notion of shared group state, and a key derivation process that ensures all group members have the cryptographic information to communicate with each other. However, we remain agnostic as to whether the "E2EE layer" on which the applications we analyze are built uses MLS, or a different form of E2EE messaging. We assume only that our E2EE infrastructure provides at least confidentiality and authenticity, though it may additionally provide deniability, forward secrecy, and post-compromise security [Beurdouche et al. 2023, §8.2]. We additionally assume that each group member knows and has agreed to the current list of all group members in the E2EE layer, another important property for private communication [Beurdouche et al. 2023, §6.1]. We briefly discuss moderation in anonymous settings in Section 6.2.

3.2 Attack Overview

In general, the malicious users' goal is to bypass the content moderation system of the application in some way. For now, consider three example attacks that will be captured by our system, which we will generalize into three adversary goals in Section 3.4.

(1) Malicious user Alice is part of a group where messages are not displayed until they are approved by a human moderator. Despite this, by altering her own application state, Alice is able to cause honest users to receive harmful messages without passing moderator approval.
(2) Malicious user Bob lacks the permission to see age-gated content, which can only be viewed by users who have a particular permission. Nonetheless, by altering his application, Bob can see the age-gated content.
(3) Malicious users Carlos and Dana collude to send each other adult content, which is banned by the Terms of Service and should have been removed by the moderator.

In Section 3.4 we will define an abstract notion of "content moderation" that applies to all of these scenarios, and define three possible ways in which the adversary might bypass moderation, which roughly correspond to generalizations of the above example attacks.

Although the quality and fairness of content moderation systems are a vital area of concern generally, for the sake of analyzing what is and isn't possible in this setting we will assume that the moderated system is exactly what the users "should" see. Our main concern is when malicious users cause the state to deviate from this idealized moderated system.

3.3 Shared State and the Moderation Function

Before defining our attacks, we must describe two key notions: the *shared state* (split into two parts, the application shared state and the shared state of the underlying E2EE communication layer), and the *moderation function*.

The *shared state* encompasses, but is not limited to, state that all parties share for the underlying E2EE messaging layer, which helps the parties' cryptographic information remain synchronized as the key is continuously evolved between messages [Barnes et al. 2023]. In addition, the shared state captures any and all shared application-layer information, including the contents and metadata of messages, settings for content annotation (warning labels, counterspeech, interstitial warnings, tags, etc.), the permissions of each user to view, create, modify, or delete their own or others' content, any ban, timeout, or suspension information, shared group metadata (e.g., group members, title, picture), and any other data shared by all users of the application. The goal is for application-layer shared state to be fully-agreed-upon by all group members at each point in time, and it will evolve over time as users send messages.

The underlying E2EE messaging layer also keeps state, both used for public *group information* (e.g., who is part of the group) and for *key management* (so users will possess the proper keys to communicate with each other, which constantly evolve as more messages are sent) [Alwen et al. 2019; Barnes et al. 2023; Cohn-Gordon et al. 2020, 2018; Marlinspike 2016]. For the underlying E2EE messaging later, state agreement is necessary in order to ensure users are able to derive compatible keys to communicate with each other.

The other notion we must introduce is the *moderation function*, which, given an input shared state and a new message sent by a sender, defines what the new shared state is supposed to be. Suppose the previously-agreed-upon shared state is $\mathsf{st}_{\mathsf{before}}$. Let the content moderation function be called f, which takes in a message x and its sender s and returns $\mathsf{st}_{\mathsf{after}} = f(\mathsf{st}_{\mathsf{before}}, x, s)$, the new shared state of the application.

Notice that this definition of "the content moderation function" actually captures quite a lot. Not only does it cover what content is shown to users (e.g., whether or not users will have profanity filtered at the application layer), but it also covers more complex notions, like what users have what permissions, checking banlists and timeouts, what users are part of the group, and so on.

We make the following additional observations about our model:

First, the shared state may still contain information that is specific to particular group members. At bare minimum, the application could simply store the information in the clear, along with a list of which group members are privileged to access it, and honest applications will only display the information to users with the appropriate privilege. This method lacks cryptographic enforcement, but is sufficient for our analysis of what is and isn't available to the attacker at the application layer.

We also make the remark that all the notion of "good" or "bad" is handled by the moderation function f, *not* the input message x. If x is a "bad" message (like a picture of gore that violates community guidelines, or an action that would require a higher permission than the sender has), it is the job of f to yield a shared state $\mathsf{st}_{\mathsf{after}} = f(\mathsf{st}_{\mathsf{before}}, x, s)$ that takes the appropriate action. For a gore picture, the application will apply its content moderation policy: it might do nothing (and wait for a moderator to take a manual action), or use an automated detection system to detect the image and apply any number of responses (a blur for users under 18, removal of the image, a warning label, etc.). For an attempted privilege escalation, the correct resulting shared state would *not* escalate the user's privilege; it should instead make no change, or inform a moderator of the attempted privilege escalation, or a similar action.

3.4 Setup and Possible Attacks

Informally, the adversary's goal is to make the content moderation "fail" somehow. If the users already all shared state $\mathsf{st}_{\mathsf{before}}$, the adversary's goal is that, after sender s sends some message x, the new shared state $\mathsf{st}^*_{\mathsf{after}}$ is *not* equal to $f(\mathsf{st}_{\mathsf{before}}, x, s)$ like it should have been. Our three settings below capture three important ways in which this might cause content moderation to fail.

We present the following informal "game" that represents the powers the adversary is allowed to use, then after that we describe what it means, in different settings, for the adversary to "win," subject to the technicalities discussed in Section 3.5.

Game 1. ***Given.*** *Shared state st_{before}, and N users in the application.*

Sending. *The adversary chooses up to $(N-1)$ users to corrupt, and chooses a sender s. If s is honest, it runs the normal application logic on x. If s is dishonest, it instead runs arbitrary code on x and sends (possibly modified) message x' over the E2EE messaging layer.*

Receiving. *The E2EE messaging layer eventually returns (y', s) to all receivers. For each receiver r, if r is honest, the normal application is run on the result. If r is dishonest, then it instead runs arbitrary code on the result. Either way, eventually the application returns y to the receiver.*

If all users are honest, $y = st_{after} = f(st_{before}, x, s)$ for all r.

In general, the adversary's goal in this game is for the receivers to have $\mathrm{st}^*_{\mathrm{after}}$ that is "invalid," that is, it differs from honest $f(\mathrm{st}_{\mathrm{before}}, x, s)$ in some way. In Section 4 we will describe specific ways in which the adversary will try to alter the state. Whether or not each of those altered states is "meaningful" depends on which users are adversarial, so we define three possible settings depending on which users are corrupted:

Setting 1 (Affect Honest User (AHU)). *In the first setting, the adversary wins if, after the game, for at least one honest user's view of st^*_{after} is not consistent with honest state $st_{after} = f(st_{before}, x, s)$.*

This attack corresponds to several content moderation failures for corrupt sender Alice and honest receiver Bob, including:

- Alice has convinced Bob's application that she has permissions that she should not have
- Alice has evaded a temporary or permanent ban (i.e., the correct new shared state $\mathrm{st}_{\mathrm{after}} = f(\mathrm{st}_{\mathrm{before}}, x, s)$ should have included Alice on a banlist, but $\mathrm{st}^*_{\mathrm{after}}$ does not reflect this)
- Alice causes Bob to view content that should have been filtered (e.g. gore)
- Alice has added a new user to the group that should not have been added (i.e., the correct new shared state $\mathrm{st}_{\mathrm{after}} = f(\mathrm{st}_{\mathrm{before}}, x, s)$ would not have changed the group membership, but $\mathrm{st}^*_{\mathrm{after}}$ does so)

We consider these kinds of attacks to be the first priority for E2EE group content moderation.

Setting 2 (Bypass Moderation Alone (BMA)). *In the second setting, one user r is malicious, and this user is not the sender. The adversary's goal is for $st^{*,r}_{after}$ to be inconsistent with honest state $st_{after} = f(st_{before}, x, s)$ for user r.*

The primary goal in this setting is for the adversary to unilaterally gain access to content that it was not supposed to have (e.g. bypass an age-gate). In some sense, this is a trivial attack—the malicious receiver can post-process her state y' received from the E2EE moderation layer however she wants. However, the *meaningful* ways in which this can happen are limited, as we will discuss further in Section 4. The primary purpose of this setting is to consider attacks in which the adversarial receiver grants herself "read" access to information that the application layer attempted to keep hidden from her. Attacks by a lone malicious user that instead alter an honest user's state are addressed in the AHU setting instead.

Setting 3 (Malicious Subgroup (MS)). *In the third setting, the sender is corrupt and at least one receiver is corrupt; the set of corrupt users is called the* malicious subgroup. *The adversary wins if, after the game, the malicious subgroup has shared new state st^*_{after} that is inconsistent with honest state $st_{after} = f(st_{before}, x, s)$.*

This corresponds to a world where the malicious subgroup wants to work together to bypass the moderation and take actions that would otherwise not be allowed. In principle, this could be applied to any shared state action including promoting user permissions and ban evasion, but our primary concern in the MS setting rather than the AHU or BMA settings is for the malicious subgroup to send content to each other that is banned by the Terms of Service while evading moderation.

These settings capture three important content moderation concerns.

3.5 Technicalities and Assumptions

We make several assumptions in our analysis. These are discussed in more detail in Appendix A, but we highlight two points: First, we assume non-anonymity at the E2EE layer—the receiving application has accurate knowledge of which sender sent a message through the E2EE channel. See Section 6.2 for a brief introduction to moderation in anonymous settings. Second, in what we will refer to as the "Non-minimal Receiver Validation Assumption," we assume that receiving devices are capable of performing at least some non-minimal validation step (i.e., the relevant portion of the content moderation function f can be run by the E2EE layer and the receiver, not fully on the sender).

We describe more details on our model's treatment of "group membership" in Appendix A.1, and more on adversaries' capabilities and goals in Appendix A.2.

4 FEASIBILITY OF ATTACKS

Our core question is: In each attack setting, in what significant ways will the adversary be able to alter the shared state? A summary of our findings is shown in Table 1.

The attacker goals shown in the rows of Table 1 were obtained by looking through the following pages that describe the moderation systems of the non-encrypted group chat applications for Slack [2023a], Discord [2023; 2023b; 2023; 2022; 2023], YouTube [2023], Facebook groups [2018], Internet Relay Chat [2000a; 2000b], and an example web forum [AliceWyman et al. 2023]. The moderation actions were flipped to determine what a successful route around that moderation tool would look like, and placed into the rows of Table 1.

For each of attacker goal, we consider whether the attacks Affect Honest Users (AHU), if they Bypass Moderation Alone (BMA), or if they yield Malicious Subgroups (MS), the settings defined in Section 3.4.

4.1 Alter state of receiving applications

This set of attacker goals essentially attempts to alter the shared state in a way that will cause content moderation to fail from the view of receiving applications. Table 1 contains six examples of such functionality. One key insight is that the level of abstraction we are discussing here, all of these actions are quite similar, as they all involve the adversary sending some kind of communication to the underlying E2EE messaging layer in an attempt to cause unwanted behavior on the receiver's application. Note that for simplicity's sake, we assume f refers to only the portion of the content moderation function that is run by the E2EE application layer and receiving client only, and does not have any functionality take place on the sender (the Non-minimal Receiver Validation Assumption).

Altered State	Adversary's Goal	Security in setting... AHU	BMA	MS	Details
Alter state of receiving applications	Privilege escalation	✓	-	(✗)	Section 4.1
	Harassment	✓	-	(✗)	
	Alter vote result	✓	-	(✗)	
	Bypass rate limit	✓	-	(✗)	
	Ban evasion	✓	-	(✗)	
	Improper edit/delete	✓	-	(✗)	
Alter state of only malicious applications	Read hidden information	-	✗	✗	Section 4.2
	Evade moderation	-	(✗)	✗	
Alter E2EE messaging state	Add/remove members	(∗)	(∗)	(∗)	Section 4.3
	Alter key derivation	(∗)	(∗)	(∗)	
Alter external state (e.g., reporting)	Falsify report	✗	✗	(✗)	Section 4.4
	Evade reporting	(∗)	(∗)	(✗)	

Legend:
[✓] App-level content moderation succeeded; attack not viable (In AHU, subject to Non-minimal Receiver Validation Assumption (Appendix A.2))
[(✗)] App-level content moderation partly succeeded; attack has limited or no impact
[✗] App-level content moderation failed; attack likely viable without additional cryptography
[(∗)] Depends on properties external to the application layer
[-] Attack not applicable in this setting

Table 1: Group moderation functionality and feasible attacks if content moderation is implemented at the application layer.

Each of these is essentially a special case of the observation that as long as $\mathsf{st}_{\mathsf{before}}$ is not corrupted, and the E2EE layer authenticates the sender s, the content moderation function f acts on honest devices to ensure that the new state, $\mathsf{st}_{\mathsf{after}}$, is valid for honest users for any message x the sender sends.

- Privilege escalation: In privilege escalation, the adversary attempts to give themselves permissions they do not have, for example, escalating themselves to "admin" status within the group with the extra powers that entails. We assume $\mathsf{st}_{\mathsf{before}}$ maintains a list of permissioned users, malicious sender s does not have the permission to promote themselves, and message x contains the logic to add s to the list of permissioned users. This can be prevented by having f check to ensure that any message granting additional permissions was sent by a sender who already has admin permissions (according to the previous shared state), rejecting $\mathsf{st}^*_{\mathsf{after}} = f(\mathsf{st}_{\mathsf{before}}, x, s)$ where this is not true.
- Harassment: A malicious sending Alice might try to spam an honest receiver with NSFW or other harassing content. There are several content moderation mechanisms that might try to stop Alice: A human moderator might need to approve Alice's messages, a client-side filter might blur Alice's messages until clicked on, or Bob might simply have requested an application-level "block" that avoids Alice's messages. The adversary wins if, despite these moderation tools, Alice is able to send the unfiltered harassing messages to Bob anyway. Since the relevant parts of f are run by the E2EE layer and receiver, any x' sent by the sender will modify the state in the usual way as $\mathsf{st}_{\mathsf{after}} = f(\mathsf{st}_{\mathsf{before}}, x', s)$.
- Alter vote result: Some content moderation mechanisms utilize various forms of voting, in which each user s sends a vote in the form of a message x_s, and the shared state tracks the votes (and voters) so far. The adversary wins if altering malicious parties' state causes the vote to be inconsistent with the honest users' votes. In this case, we require the state to keep a list of votes and voters so far, and f to reject (or otherwise handle) any duplicate votes cast. Honest voters will use the honest state $\mathsf{st}_{\mathsf{after}} = f(\mathsf{st}_{\mathsf{before}}, x_s, s)$ which correctly handles any duplication of votes, though malicious users may continue using a different state $\mathsf{st}^*_{\mathsf{after}}$.
- Rate limit evasion: Many chat applications implement rate limits for users who are sending messages too quickly. Here the adversary wins if they are able to change the rate limit, or send messages more quickly than they are supposed to. If the honest shared state includes the timestamp of the sender's last message, it can be compared with the time the message is received by the receiver. This means f can reject a newly received message from s until the rate limit has passed. (This elides a more formal delicate treatment of time, but is a proof of concept to illustrate that this can be handled at the application layer.)
- Ban evasion: The sender is prohibited from sending to one or more honest users, and this fact is recorded in the honest shared state. Because the sender is known and authenticated via the E2EE layer, the receiver's implementation of f can discard messages sent from the banned user.
- Improper edit/delete: Most group chat applications limit the ability of normal users to edit or delete others' content. If a malicious sender is able to do this by sending a message, this would be a successful attack. However, because the sender is known, and the permissions are stored in $\mathsf{st}_{\mathsf{before}}$, f can reject any requests to alter the state in unauthorized ways.

Under the Non-minimal Receiver Validation assumption, and the assumption of non-anonymity (discussed briefly in Section 3.5 and in detail in Appendix A.2), these attacks should not be able to cause any invalid state for honest users (AHU). These properties

can all be validated using fairly straightforward application logic we encapsulate in f. Thus, these are all not a concern in the AHU setting.

These are all irrelevant in the BMA setting, because they all require a malicious sender to cause a change to another device, rather than considering what can be changed for the malicious user herself. Privilege escalation is slightly different, because in principle a user could alter her own privileges on her own device. However, since those privileges would not have any impact on other users' devices, the only purpose would be to gain access to information that the malicious user should not have. We cover this case under "Read hidden information" in Section 4.2 instead.

Application-only content moderation is in some sense vulnerable to these attacks in the Malicious Subgroup (MS) setting: a malicious subgroup *can* choose to ignore the validation performed by honest devices. For example, if malicious Alice attempts to promote herself to admin privileges, malicious receivers can choose to go along with this. However, this will not leak out to honest users unless the underlying E2EE messaging layer is no longer agreed upon (which we consider under "Alter key derivation" in Section 4.3).

This demonstrates that, subject to mild assumptions, honest users' state cannot be tampered with even if content moderation is performed at the application layer. Although it is possible for a malicious subgroup to cause trouble within their subgroup, this bad behavior does not leak out to honest users who are running the honest application.

This is a very positive result, because it indicates that many content moderation tasks can already be done in end-to-end encryption utilizing only the cryptographic infrastructure of the underlying authenticated E2EE layer, and the additional receiver validation needed beyond that need not be cryptographically strong.

4.2 Alter state of only malicious applications

We next consider the possibilities for evading content moderation that only affect malicious users (thus, AHU is not in scope). There are two primary goals for adversaries in this setting: read information that should be hidden from them, and evade moderation.

Consider users who wish to read hidden information: Here, there is some set of information that is being shared within the shared state. For example, perhaps only users with a particular privilege may view particular content (e.g. to view NSFW content you must have a flag indicating that you are over 18 years old). The main goal of the BMA setting is for a lone malicious user to view that information anyway. Because the application is relying on logical rather than cryptographic barriers to viewing content, this attack is feasible. It also remains feasible in the MS setting where there are multiple malicious users. Most E2EE implementations assume that all parties in the group should know who the group members are (even at the application layer, see [Beurdouche et al. 2023, §8.4.3.1] for a discussion of this assumption in E2EE, and see Section 6.2 for more on anonymous applications) and assume that all parties in the group should be able to read all content. In the context of E2EE, encryption should be used to hide information from malicious adversaries.

Now turning to users who wish to evade moderation: the main goal of adversaries in a Malicious Subgroup (MS) is to send content to each other that would normally be detected or blocked by the moderation function, while avoiding either detection of the content as problematic, or the blocking of that problematic content, or both. As we discuss in prior work [Scheffler and Mayer 2023, §5.3] and in Section 6.1, the ability to do this requires at least some form of honesty assumption by the clients, who if nothing else can perform their own key exchange over the existing channel and send each other unmoderated content that is encrypted in their new channel before inputting it to the application logic. Even if one assumes at least one party behaves honestly, this is still an incredibly difficult technical task that also has heavy implications for mass surveillance capabilities that we delve into in detail in prior work [Scheffler et al. 2023; Scheffler and Mayer 2023]. Essentially, this case reduces down to the same difficult problem that has been subject to major debate and discussion with the proposal of some recent legislation [European Union 2022; United Kingdom 2021; US Senate 2023].

This goal is something we also consider in the BMA setting, but in this case, the malicious user would only be able to access unmoderated content sent by honest users or herself (since there is only one malicious user). We believe this is a much less serious scenario than the MS setting, and in some cases could even be considered a feature (e.g., turning off an over-active spam filter).

4.3 Alter E2EE messaging layer state or key derivation

Malicious parties could also attempt to alter the shared state or key derivation process of the E2EE messaging layer. For example they could attempt to add an additional group member without confirmation by the group, remove an existing member improperly, or alter the key derivation process in some way. As mentioned in Appendix A.1, the E2EE messaging layer usually does not have baked-in notions of permissions, but it *does* have some notion of shared (public) state that must be agreed upon to communicate [Barnes et al. 2023, §8.1]. In the MLS specification, this contains cryptographic information used for key derivation and ensuring that the message history (at this layer) agrees for all group members. The extent to which attacks that alter this layer (e.g. improperly add a group member) will work largely depends on the underlying E2EE messaging layer infrastructure rather than the application, and so we do not discuss it further here.

4.4 Alter external state

The last question we consider is whether the user can cause content moderation to when that moderation involves an interaction between the user's application and some external entity who is not part of the E2EE group—e.g. an external banlist, or a service that accepts user reports.

First, consider a malicious user whose goal is to have the external entity accept false information. This could be "framing" a user for sending some content or an attempt to impersonate a moderator to an external entity. This differs from the attacks described in Section 4.1 because they may involve reporting what *other* users have done, to an *external* entity who cannot necessarily access plaintext. Unlike in Section 4.1, external entity does not necessarily have the ability to validate the malicious user's claims.

The success of attacks here are partly dependent on the implementation of both the application and the E2EE messaging layer, but we mark these as successful attacks because without at least some cryptography, it will be possible for the user to falsify information provided to an entity external to the E2EE group. Because of this, we deem the attack to function in all three settings (AHU, BMA, and MS), though the cryptography used to verify this information need not be the same used in the E2EE messaging layer.

One cryptographic solution to prevent users from creating falsified reports is *message franking* [Facebook 2016; Issa et al. 2022; Tyagi et al. 2019a]. However, it should be noted that the non-repudiation property that comes with message franking alters the deniability property that may be provided by the underlying E2EE messaging layer [Tyagi et al. 2019b, §B]. See Section 6.1 for more details.

The second attack is to evade an action taken by the external entity, or take some action that prevents honest users' from interacting correctly with the external entity. For example, honest user Bob might add malicious user Alice to an external banlist; Alice can evade the ban by preventing Bob from doing so or taking herself off the banlist. The success of this attack is largely based on the implementation itself.

5 CURRENT IMPLEMENTATIONS OF GROUP MODERATION IN E2EE

In contrast to the previous section, which discussed the potential for group moderation that would mimic that of popular non-E2EE group applications, in this section we discuss the current state of moderation in existing end-to-end encrypted group applications, and discuss some more complex cryptographic tools that could be applied to achieve additional moderation properties.

Table 2 shows the moderation functionality implemented in some popular E2EE messaging applications. To build this table, we used documentation for Signal [2023; 2023], WhatsApp [2022; 2023; 2023], iMessage [2021b; 2022a], and two applications built on top of the Matrix communication protocol: Element [2022; 2023b; 2023a] and a generic chat application using only the Synapse server tools [2023a; 2023b].

We make several observations about these results:

Permissions are already implemented. We immediately observe that the moderation implemented in these settings accomplishes many (though not all) of the goals described in Section 3 for non-encrypted group applications. Signal, WhatsApp, and both Matrix protocols all implement some notion of permissions similar to how we describe in Sections 3 and 4.

Limited support for deletion of others' content. We observe that in principle, admins or other permissioned users can take action on others' content, as is shown by the case of Element. (We point out that allowing non-permissioned users to edit others' content is rare even in the unencrypted setting.) The other applications examined do not allow any users, even admins or moderators, to delete others' content. However, they do have options for individual receivers to remove messages in their own local view on their devices.

Highly restricted editing. In principle, we expect that moderator actions on others' content (e.g. age-gating, blurring, or appending to it) are all functionalities that could be implemented in E2EE chat. However, we do not see this implemented in any of the applications we examined.

Almost all of the applications examined provide users with the ability to edit their own messages, however some (Signal, iMessage, and Element) show an un-alterable edit history.

We make the qualitative observation that being able to view an un-altered "full history" of the group has been a concern among developers and users of these applications (see e.g. open conversations in Signal's code repository [Signal-Desktop 2018]).

Wide variety in functionality. We also observe that "E2EE groups" span a very large range of use cases. Apple iMessage is essentially a group version of one-to-one chat—all users have the same privileges, and it seems to be tailored to the use case where a small group of trusted friends are using it to text. Signal is similar, but with slightly different permissions options. On the other side of the scale, Element pitches itself as more of an alternative to Slack, with threads, multiple rooms, productivity tool integration, and significantly more content moderation tools [Element 2023a].

6 OTHER APPROACHES TO MODERATION IN E2EE

Although the focus of this work is on general group moderation tasks (i.e., not specific to E2EE communications) and understanding their implementation in E2EE group settings, some additional tools exist for content moderation that are specific to the E2EE context. We describe these in greater detail in our prior work [Scheffler and Mayer 2023] and we additionally briefly review them here in Section 6.1. We also briefly review methods for moderation in anonymous systems in the academic literature in Section 6.2.

6.1 Other Moderation Tools in E2EE

Message Franking. Message franking provides cryptographic verification to user reports: Specifically, it ensures that malicious Alice cannot forge a report on content that she claims was sent by honest user Bob, that Bob in fact never sent. (Separately, it also ensures that Bob cannot repudiate content that he did send, though this has subtly different consequences on deniability depending on the key setup [Tyagi et al. 2019a].) Although message franking is popular in academic literature [Issa et al. 2022; Tyagi et al. 2019a], it has achieved very little industry adoption [Facebook 2016].

Source Tracking and Traceback. Our model required that the most recent sender of a message be known—but it did not say anything about prior senders of the message. Motivated by the ability to locate the original senders of disinformation, Peale et al. [2021], Issa et al. [2022], and Tyagi et al. [2019c] all implemented some form of *source tracking* or *traceback*, cryptographically storing the original source, or forwarding path/tree, of a message that had been forwarded along multiple "hops." As far as we know, these have not been implemented in any widely-deployed E2EE system.

Methods for moderating encrypted content without plaintext. As we discussed in Section 4.2, the problem of moderating encrypted content between multiple parties without access to plaintext is extremely difficult, and often impossible. The first problem is that

		In this E2EE application, which users can take the action:				
Category	**Action**	**Signal**	**WhatsApp**	**iMessage**	**Element (Matrix)**	**Synapse (Matrix)**
Sending	Send messages	Permission	Permission	All	Permission	External[1]
	Report to in-group admin	Out-of-band	Room setting	Out-of-band	All	External[1]
Editing	Edit others' messages	None	None	None	None	None
	Delete others' messages	None[2]	None[2,3]	None	Permission	Admin
	Edit own messages	All[6]	All[4]	All[4,6]	All[6]	None
	Delete own messages	All[4,5]	All[4]	All[4]	All[5]	Admin[5]
Group	Kick group members	Admin	Admin	All	Permission	Admin
	Add group members	Permission	Admin	All	Room setting	Room setting
External	Report outside group	Out-of-band	All[7]	Out-of-band	External[1]	External[1]

[**All**] Action can be performed by all group members
[**None**] No group member can take the action
[**Admin**] Action can be performed by all group admins
[**Permission**] Action can be performed by group members with a particular permission (even non-admins)
[**Room setting**] Admins (or those with permission) can allow or disallow all group members in the room from taking the action
[**Out-of-band**] User is instructed to handle the action via a different channel (not guaranteed to exist for all groups)
[**External**] Ability for action to be taken by making use of external (not built in) tools

Notes:
1: Can be achieved using additional moderation code such as Mjolnir [matrix-org 2022, 2023] or Draupnir [The Draupnir Project 2023]
2: Can remove others' messages in own local view
3: Functionality currently in beta [Hardwick 2022]
4: Only for a limited time after sending
5: Shows deletion
6: Shows edit history
7: Last five messages from whole group

Table 2: Group moderation functionality implemented in common E2EE messaging applications

it is extremely difficult to perform moderation *of content* without knowledge of plaintext at all. One approach is to moderate using only metadata, like message length, frequency, source, or destination, which can sometimes provide enough information for moderation decisions without processing content at all. Another approach is to process content in some *oblivious* way, either using privacy-preserving computation or relying on on-device analysis by an honest user. Our previous work [Scheffler and Mayer 2023] performed a full literature search and systematization of all moderation systems we could identify that analyzed content in encrypted settings.

However, the second problem—an observation we made in that work—was that for those content moderation systems to detect (or respond to) problematic content at all, at least one party in the computation *must* be honest, or else two malicious parties could simply build their own layer of encryption on top of the existing E2EE application, and bypass moderation by doing so (since no content moderation system should be able to decrypt a ciphertext built by the adversary). This honesty requirement is even stricter than the cryptographic notion of honesty: even if the parties follow the cryptographic protocol correctly, they can input information into that protocol that is not the content itself. By inputting a ciphertext into the moderated E2EE protocol, the parties can evade any content moderation system built (or else, that same content moderation system could break modern cryptography).

This aligns with the difficulty we observed with handling the Malicious Subgroup attack category in Section 4.2. Without at least one honest party in the loop, moderation options are quite limited.

6.2 Moderating Anonymous Systems

Throughout most of this work, we assumed the sender was known. Dealing with anonymous senders makes application-level (and any) moderation of those users more difficult, but not impossible. In this section we describe some methods for reporting or banning users in anonymous networks.

However, the cryptography literature provides a number of frameworks for "accountable anonymity" in which actions can be taken against problematic users even without the server learning the identity of the user directly or indirectly via linking. As the anonymous network Tor [Dingledine et al. 2004] gained adoption in the late 2000s, problematic Tor users became issues not only for web servers but also for non-problematic Tor users, who found that the web servers solved their problematic Tor problems by IP-blocking Tor exit nodes in their entirety. Nym [Holt and Seamons 2006] was introduced as a pseudonym system meant to allow websites to block individual pseudonymous Tor users rather than relying on IP-based blocking, hoping that this would cause services to remove their IP-based bans of Tor exit nodes. This was quickly improved into a computational anonymity guarantee by Nymble [Johnson et al. 2007] and a variety of improvements over the years [Henry and Goldberg 2011a; Henry et al. 2010; Lofgren and Hopper 2012], though Nymble users could still be de-anonymized by either one trusted party or a group of parties with a non-collusion requirement. These works were primarily created for the purpose of allowing websites to block specific problematic Tor users as an alternative to blocking entire Tor exit nodes by IP address [Henry

and Goldberg 2011a; Henry et al. 2010; Holt and Seamons 2006; Johnson et al. 2007; Lofgren and Hopper 2012], though more recent work has brought this idea to sender-anonymous messaging [Tyagi et al. 2022].

Tsang et al.'s 2010 paper introduced the "BLacklistable Anonymous Credential" (BLAC) [2010] which has since been improved in a variety of ways since [Au and Kapadia 2012; Au et al. 2011; Henry and Goldberg 2013; Huang et al. 2021; Lofgren and Hopper 2011; Lueks et al. 2016; Nakanishi and Kanatani 2020; Wang et al. 2017; Xi and Feng 2014; Yu et al. 2012]). The core idea combines a blacklist with a ticketing system, such that service providers could add the tickets of problematic users to a blacklist (temporarily or permanently), and that ticket-holder would be banned from the service as long as one of their tickets was on the blacklist, even when using different (non-linkable) tickets.

Other works also propose different methods for blocking [Au and Kapadia 2012; Henry and Goldberg 2011b; Rosenberg et al. 2022; Xu et al. 2012], tracing back [Backes et al. 2014], or punitively de-anonymizing [Daza et al. 2021; Köpsell et al. 2006; Yang et al. 2017] anonymous users, including some work on intentional de-anonymization in cryptocurrencies where the anonymity comes from "joins" or "rings" of transactions [Keller et al. 2021].

A related line of work also focuses on halting high-traffic denial of service attacks in anonymous networks by meting out a certain number of "tokens" to anonymous users [Davidson et al. 2018; Döpmann et al. 2021; Mani and Goldberg 2021], a form of rate-limiting on users whose problematic actions induce high traffic. An older survey on anonymous blacklisting systems was performed by Henry and Goldberg [2011b].

Very recently, blocking anonymous users has also been achieved in Signal's "sealed sender" system [Tyagi et al. 2022], and Issa et al. [2022] build asymmetric message franking in sealed sender.

Additionally, at least two works proposed cryptographic mechanisms allowing individuals to *anonymously report* users [Kuykendall et al. 2019; Rajan et al. 2018], potentially revealing the identities of the reporters once a threshold of reports was reached. In [Rajan et al. 2018], the identity was revealed to a lawyer affiliated with the service; in [Kuykendall et al. 2019] the identity was revealed to the reporting users.

7 CONCLUSION

This work sets out to explore what group moderation tasks can be performed in E2EE settings, especially at the application layer.

By using our model described in Section 3, we find that the many of the most common group content moderation tasks—those that ensure that honest users have a positive user experience—can be implemented at the application layer on top of an existing E2EE protocol (Section 4). We believe this adds merit to the notion that some forms of content moderation are compatible with E2EE. We also emphasize that these moderation forms all utilize existing notions of trust, do not involve any disclosure of information to a party that was not already trusted to hold plaintext, and are sufficient to prevent honest group members from being affected by malicious users.

We see several of those tools implemented in current E2EE group chat applications available today, but we also note that many E2EE chat applications do not provide these moderation tools (Section 5).

However, the challenge of preventing multiple malicious parties from colluding and evading content moderation (Section 4.2) in which the group E2EE challenges are largely the same as in the one-to-one E2EE setting.

Although it is not a panacea, we see group moderation in E2EE as a helpful tool that is compatible with existing notions of privacy in E2EE, and we hope to see more research and implementation of these systems in the future.

ACKNOWLEDGEMENTS

This work was supported by the Center for Information Technology Policy at Princeton University, the Internet Policy Research Initiative at the Massachusetts Institute of Technology, and a cryptography research grant from Ripple. We also thank anonymous reviewers for their insightful and helpful feedback.

A ASSUMPTIONS AND TECHNICAL DETAILS

This section contains more technical details of our modeling and the assumptions we made in Sections 3 and 4.

A.1 Application Users versus Messaging Layer Group Membership

Although the goals of content moderation discussed here are often agnostic to the underlying technical implementation, there are several implementation details of what it means to be a "group member" on the E2EE message layer, that merit mentioning since they can misalign with the concept of a "user" at the application layer.

Agreed group membership is key for technical implementations of end-to-end encryption. From the point of view of the E2EE messaging layer, a particular key-holder is either in the group, or out [Barnes et al. 2023; Beurdouche et al. 2023]. This has a couple of consequences.

First, although many group chat applications only allow administrators or privileged users to add or remove members of a group, in the E2EE messaging layer, the parties must all agree on who the group members are for the key derivation process to work. If this fails, and a subset of users assumes a different set of group members than the others, the entire key derivation process is likely to be mangled.

Second, one large component of group moderation in applications like Slack, Discord, or online forums is the question of managing users' access to different "channels" or "threads" within one larger community. If this access restriction is to be enforced cryptographically, then the most straightforward way to manage this would be to have a separate messaging-layer group for each channel or thread. This has some efficiency challenges—in particular the user would need to store different shared state (e.g. key material) for each channel. But the other straightforward alternative, of performing the restriction at the application layer and maintaining one large shared state object over the entire community, may allow the adversary to read that private information. Our analysis in Section 3 is built on the level of one messaging-layer group.

Finally, in this work, we remain agnostic as to whether a "group member" refers to a particular client device, or a particular user (who may possess several client devices in the group, say, a cellphone and a desktop computer). The difference is not important for this work, but it is important for some components of an application-layer moderation system built on top of an E2EE messaging layer, because a user may possess several client devices. For example, when a user is banned, the likely intended result was to ban all of the user's devices, which may correspond to multiple "group members" in the messaging-layer protocol.

Note that we do not consider anonymity except in Section 6.2.

A.2 Assumptions in our Analysis

Our model in Section 3 the following assumptions:

First: We assume that a "malicious" user is capable of modifying their application client in any way, but cannot modify honest users' applications except by sending normal messages through the E2EE messaging layer.

Second: all communication occurs through the normal group E2EE messaging layer, honest users will not accept any adversarial messages sent through any other channel.

Third: An adversarial sender wins only if it can create a state that is inconsistent with *any* honest action of the sender, and our notation "x" is a stand-in for any such valid input. This choice greatly simplifies notation, but is an abuse of notation. This property is needed to ensure that the adversary cannot score a trivial win by swapping out what would have been the honest output on one input with the honest output on another input. In more detail, the adversary can pick "real" message x_0, but then alter the message to $x_1 \neq x_0$ before applying the (honest) application logic. If this is the case, then all parties have a shared new state $\mathsf{st}_{\mathsf{after}} = f(\mathsf{st}_{\mathsf{before}}, x_1, s)$ that is *consistent* with the world where the sender honestly sent message x_1 and submitted it to the content moderation function as normal. However, this message x_1 may not be the message x_0 that the sender actually sent. In reality, we only ask that the new shared state is *consistent* with some message x_1 which the sender could have chosen and sent honestly. This is a familiar notion in cryptography literature (e.g. [Ishai et al. 2007]) but will add a confusing additional layer for our analysis here. Thus, we make the simplifying assumption that for malicious sender s, if there exists some input x_1 consistent with the (deterministic) output $\mathsf{st}^*_{\mathsf{after}}$ of the function such that $\mathsf{st}^*_{\mathsf{after}} = f(\mathsf{st}_{\mathsf{before}}, x_1, s)$ is, for all intents and purposes, the result of running the honest system on input x_1, we treat the adversary's choice of x as that x_1.

Fourth: As is standard in cryptography, we limit our attention to adversaries that act in polynomial-time in the security parameter of the underlying E2EE scheme (and can therefore not break the cryptographic guarantees in the E2EE layer). We limit our attention to deterministic adversaries.

Fifth: We assume that the E2EE messaging layer is non-anonymous, so receiving applications know which sender sent a particular message (even if they do not display this information to the user), and that all group members are known. See Section 6.2 for more in the anonymous setting.

Sixth: We assume f is deterministic for simplicity.

Seventh: We assume that, in the process of processing a message x sent by a sender, the application always makes exactly one call to the underlying E2EE messaging layer, which takes in message x' from sender s and returns (y', s) to all receivers.

Finally, for the AHU setting only, we make one additional assumption: "Non-minimal Receiver Validation.": Content moderation does not solely happen on the sender's device, and that receivers will (non-cryptographically) validate requests. The purpose of this assumption is to prevent very trivial attacks that would arise if all the content moderation processing was performed by the sender—a malicious sender could just ignore that moderation step before sending. For example, in WhatsApp groups, users can be promoted to "admin" status by other admins. (The first admin is the creator of the group.) The assumption states that if non-admin user Alice sends a message promoting herself to admin status, this is rejected by the validation code of the receiver. (In contrast, if admin Bob sent a message promoting Alice to admin, this passes the validation process.) We believe this is a reasonable assumption—it is easy to implement non-cryptographically, and is common-sense from a security standpoint.

REFERENCES

AliceWyman, Chris Ilias, Madasan, guigs, and Artist. 2023. Forum Moderation Guidelines. Mozilla. https://support.mozilla.org/en-US/kb/moderation-guidelines

Joël Alwen, Sandro Coretti, and Yevgeniy Dodis. 2019. The Double Ratchet: Security Notions, Proofs, and Modularization for the Signal Protocol. In *Advances in Cryptology – EUROCRYPT 2019*, Yuval Ishai and Vincent Rijmen (Eds.). Springer International Publishing, Cham, 129–158.

Amazon Web Services. 2023. wickr. https://wickr.com/

Apple. 2021a. iMessage Security Overview. https://support.apple.com/guide/security/imessage-security-overview-secd9764312f/1/web/1

Apple. 2021b. Name a group text message on your iPhone, iPad, or iPod touch. https://support.apple.com/en-us/HT211301

Apple. 2022a. Add and remove people in group text messages on your iPhone or iPad. https://support.apple.com/en-us/HT211302

Apple. 2022b. Advanced Data Protection for iCloud. https://support.apple.com/guide/security/advanced-data-protection-for-icloud-sec973254c5f/web

Man Ho Au and Apu Kapadia. 2012. PERM: practical reputation-based blacklisting without TTPS. In *ACM CCS 2012: 19th Conference on Computer and Communications Security*, Ting Yu, George Danezis, and Virgil D. Gligor (Eds.). ACM Press, Raleigh, NC, USA, 929–940. https://doi.org/10.1145/2382196.2382294

Man Ho Au, Patrick P. Tsang, and Apu Kapadia. 2011. PEREA: Practical TTP-free revocation of repeatedly misbehaving anonymous users. *ACM Trans. Inf. Syst. Secur.* 14, 4 (2011), 29:1–29:34. https://doi.org/10.1145/2043628.2043630

Michael Backes, Jeremy Clark, Aniket Kate, Milivoj Simeonovski, and Peter Druschel. 2014. BackRef: Accountability in Anonymous Communication Networks. In *ACNS 14: 12th International Conference on Applied Cryptography and Network Security (Lecture Notes in Computer Science, Vol. 8479)*, Ioana Boureanu, Philippe Owesarski, and Serge Vaudenay (Eds.). Springer, Heidelberg, Germany, Lausanne, Switzerland, 380–400. https://doi.org/10.1007/978-3-319-07536-5_23

Richard Barnes, Benjamin Beurdouche, Raphael Robert, Jon Millican, Emad Omara, and Katriel Cohn-Gordon. 2023. The Messaging Layer Security (MLS) Protocol. RFC 9420. https://doi.org/10.17487/RFC9420

Benjamin Beurdouche, Eric Rescorla, Emad Omara, Srinivas Inguva, and Alan Duric. 2023. *The Messaging Layer Security (MLS) Architecture.* Internet-Draft draft-ietf-mls-architecture-11. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-mls-architecture/11/ Work in Progress.

Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. 2020. A Formal Security Analysis of the Signal Messaging Protocol. In *Journal of Cryptology*. Springer, 1914–1983. https://doi.org/10.1007/s00145-020-09360-1

Katriel Cohn-Gordon, Cas Cremers, Luke Garratt, Jon Millican, and Kevin Milner. 2018. On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guarantees. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) *(CCS '18)*. Association for Computing Machinery, New York, NY, USA, 1802–1819. https://doi.org/10.1145/3243734.3243747

D. 2023. Safer Messaging on Discord. Discord. https://support.discord.com/hc/en-us/articles/115000068672

Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. 2018. Privacy Pass: Bypassing Internet Challenges Anonymously. *Proceedings on Privacy Enhancing Technologies* 2018, 3 (July 2018), 164–180. https://doi.org/10.1515/popets-2018-0026

Vanesa Daza, Abida Haque, Alessandra Scafuro, Alexandros Zacharakis, and Arantxa Zapico. 2021. Mutual Accountability Layer: Accountable Anonymity within Accountable Trust. Cryptology ePrint Archive, Report 2021/596. https://eprint.iacr.org/2021/596.

Roger Dingledine, Nick Mathewson, and Paul F. Syverson. 2004. Tor: The Second-Generation Onion Router. In *USENIX Security 2004: 13th USENIX Security Symposium*, Matt Blaze (Ed.). USENIX Association, San Diego, CA, USA, 303–320.

Discord. 2023a. Discord - Official Site. https://discord.com/

Discord. 2023b. Server Settings. Server settings on a newly created fresh Discord server with default settings.

Discord Safety Center. 2023. Discord Moderator Academy. https://discord.com/moderation/4405266071063-100-An-Intro-to-the-DMA

Christoph Döpmann, Valentin Franck, and Florian Tschorsch. 2021. Onion Pass: Token-Based Denial-of-Service Protection for Tor Onion Services. In *2021 IFIP Networking Conference (IFIP Networking)*. IEEE, 1–9.

Element. 2022. Get started with Element. https://static.element.io/pdfs/element-user-guide.pdf

Element. 2023a. An app for productivity. https://element.io/app-for-productivity

Element. 2023b. Managing a Room: Room Settings. https://ems-docs.element.io/books/element-support/page/managing-a-room-room-settings

European Union. 2022. Regulation of the European Parliament and of the Council laying down rules to prevent and combat child sexual abuse. https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=COM%3A2022%3A209%3AFIN

Facebook. 2016. Messenger Secret Conversations: Technical Whitepaper. https://about.fb.com/wp-content/uploads/2016/07/messenger-secret-conversations-technical-whitepaper.pdf

Facebook. 2023. Connect with people who share your interests. https://www.facebook.com/groups/discover/

Eric Goldman. 2021. Content moderation remedies. *Michigan Technology Law Review (Forthcoming)* (2021).

Google. 2022. *Messages End-to-End Encryption Overview*. https://www.gstatic.com/messages/papers/messages_e2ee.pdf

Tim Hardwick. 2022. WhatsApp Group Admins Can Delete Any Message for Everyone in Latest Beta *(MacRumors)*. https://www.macrumors.com/2022/08/22/whatsapp-group-admin-delete-messages-everyone/

Ryan Henry and Ian Goldberg. 2011a. Extending Nymble-like Systems. In *2011 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, Berkeley, CA, USA, 523–537. https://doi.org/10.1109/SP.2011.17

Ryan Henry and Ian Goldberg. 2011b. Formalizing Anonymous Blacklisting Systems. In *2011 IEEE Symposium on Security and Privacy*. IEEE Computer Society Press, Berkeley, CA, USA, 81–95. https://doi.org/10.1109/SP.2011.13

Ryan Henry and Ian Goldberg. 2013. Thinking inside the BLAC box: smarter protocols for faster anonymous blacklisting. In *Proceedings of the 12th annual ACM Workshop on Privacy in the Electronic Society, WPES 2013, Berlin, Germany, November 4, 2013*, Ahmad-Reza Sadeghi and Sara Foresti (Eds.). ACM, 71–82. https://doi.org/10.1145/2517840.2517855

Ryan Henry, Kevin J. Henry, and Ian Goldberg. 2010. Making a Nymbler Nymble Using VERBS. In *PETS 2010: 10th International Symposium on Privacy Enhancing Technologies (Lecture Notes in Computer Science, Vol. 6205)*, Mikhail J. Atallah and Nicholas J. Hopper (Eds.). Springer, Heidelberg, Germany, Berlin, Germany, 111–129. https://doi.org/10.1007/978-3-642-14527-8_7

Jason E Holt and Kent E Seamons. 2006. Nym: Practical pseudonymity for anonymous networks. *Internet Security Research Lab Technical Report* 4 (2006), 1–12.

Sharon Huang, Subodh Iyengar, Sundar Jeyaraman, Shiv Kushwah, Chen-Kuei Lee, Zutian Luo, Payman Mohassel, Ananth Raghunathan, Shaahid Shaikh, Yen-Chieh Sung, and Albert Zhang. 2021. *DIT: De-Identified Authenticated Telemetry at Scale*. https://scontent-lga3-2.xx.fbcdn.net/v/t39.8562-6/246534149_588854725718321_8923613326138589821_n.pdf

Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. 2007. Zero-Knowledge from Secure Multiparty Computation. In *Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing* (San Diego, California, USA) *(STOC '07)*. Association for Computing Machinery, New York, NY, USA, 21–30. https://doi.org/10.1145/1250790.1250794

Rawane Issa, Nicolas Alhaddad, and Mayank Varia. 2022. Hecate: Abuse Reporting in Secure Messengers with Sealed Sender. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2335–2352. https://www.usenix.org/conference/usenixsecurity22/presentation/issa

Peter C. Johnson, Apu Kapadia, Patrick P. Tsang, and Sean W. Smith. 2007. Nymble: Anonymous IP-Address Blocking. In *PET 2007: 7th International Symposium on Privacy Enhancing Technologies (Lecture Notes in Computer Science, Vol. 4776)*, Nikita Borisov and Philippe Golle (Eds.). Springer, Heidelberg, Germany, Ottawa, Canada, 113–133. https://doi.org/10.1007/978-3-540-75551-7_8

Christophe Kalt. 2000a. Internet Relay Chat: Channel Management. RFC 2811. https://doi.org/10.17487/RFC2811

Christophe Kalt. 2000b. Internet Relay Chat: Client Protocol. RFC 2812. https://doi.org/10.17487/RFC2812

Sarthak Kapoor. 2023. What privileges do group admins enjoy on WhatsApp? *(NewsBytes)*. https://www.newsbytesapp.com/news/science/what-all-can-a-whatsapp-admin-do/story

David A Kaye. 2019. *Speech police: The global struggle to govern the Internet*. Columbia Global Reports.

Patrik Keller, Martin Florian, and Rainer Böhme. 2021. Collaborative deanonymization. In *International Conference on Financial Cryptography and Data Security*. Springer, 39–46.

Gail Kent. 2018. Why does WhatsApp Enable End-to-End Encryption? https://about.fb.com/news/2018/05/end-to-end-encryption/

Stefan Köpsell, Rolf Wendolsky, and Hannes Federrath. 2006. Revocable Anonymity. In *Emerging Trends in Information and Communication Security, International Conference, ETRICS 2006, Freiburg, Germany, June 6-9, 2006, Proceedings (Lecture Notes in Computer Science, Vol. 3995)*, Günter Müller (Ed.). Springer, 206–220. https://doi.org/10.1007/11766155_15

Benjamin Kuykendall, Hugo Krawczyk, and Tal Rabin. 2019. Cryptography for #MeToo. *Proceedings on Privacy Enhancing Technologies* 2019, 3 (July 2019), 409–429. https://doi.org/10.2478/popets-2019-0054

Librarian. 2022. Setting Up Permissions FAQ. Discord. https://support.discord.com/hc/en-us/articles/206029707-How-do-I-set-up-permissions-

Librarian. 2023. AutoMod FAQ. Discord. https://support.discord.com/hc/en-us/articles/4421269296535

Peter Lofgren and Nicholas Hopper. 2011. FAUST: efficient, TTP-free abuse prevention by anonymous whitelisting. In *Proceedings of the 10th annual ACM workshop on Privacy in the electronic society, WPES 2011, Chicago, IL, USA, October 17, 2011*, Yan Chen and Jaideep Vaidya (Eds.). ACM, 125–130. https://doi.org/10.1145/2046556.2046572

Peter Lofgren and Nicholas Hopper. 2012. BNymble: More Anonymous Blacklisting at Almost No Cost (A Short Paper). In *FC 2011: 15th International Conference on Financial Cryptography and Data Security (Lecture Notes in Computer Science, Vol. 7035)*, George Danezis (Ed.). Springer, Heidelberg, Germany, Gros Islet, St. Lucia, 268–275.

Wouter Lueks, Maarten H. Everts, and Jaap-Henk Hoepman. 2016. Vote to Link: Recovering from Misbehaving Anonymous Users. In *Proceedings of the 2016 ACM on Workshop on Privacy in the Electronic Society, WPES@CCS 2016, Vienna, Austria, October 24 - 28, 2016*, Edgar R. Weippl, Stefan Katzenbeisser, and Sabrina De Capitani di Vimercati (Eds.). ACM, 111–122. http://dl.acm.org/citation.cfm?id=2994634

Akshaya Mani and Ian Goldberg. 2021. ZXAD: High-volume Attack Mitigation for Tor. In *Proceedings of the 20th Workshop on Workshop on Privacy in the Electronic Society*. 1–16.

Moxie Marlinspike. 2016. The Double Ratchet Algorithm. https://signal.org/docs/specifications/doubleratchet/

Matrix. 2019. [matrix] An open network for secure decentralized communication. https://matrix.org

Matrix. 2023a. Community Moderation. https://matrix.org/docs/communities/moderation/

Matrix. 2023b. Matrix Specification v1.9 (Latest). https://spec.matrix.org/v1.9/

matrix-org. 2022. Moderator's guide to Mjolnir (bot edition). https://github.com/matrix-org/mjolnir/blob/main/docs/moderators.md

matrix-org. 2023. Mjolnir. https://github.com/matrix-org/mjolnir/

Mehvish. 2018. The Complete Guide to Facebook Group Admin Roles and Responsibilities. https://www.guidingtech.com/facebook-group-admin-roles/

Meta. 2022. How technology detects violations. https://transparency.fb.com/enforcement/detecting-violations/technology-detects-violations/

Mozilla. 2023. Community Hub. https://support.mozilla.org/en-US/community/

Toru Nakanishi and Takeshi Kanatani. 2020. Efficient blacklistable anonymous credential system with reputation using a pairing-based accumulator. *IET Inf. Secur.* 14, 6 (2020), 613–624. https://doi.org/10.1049/iet-ifs.2018.5505

Charlotte Peale, Saba Eskandarian, and Dan Boneh. 2021. Secure Complaint-Enabled Source-Tracking for Encrypted Messaging. In *CCS '21: 2021 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event, Republic of Korea, November 15 - 19, 2021*, Yongdae Kim, Jong Kim, Giovanni Vigna, and Elaine Shi (Eds.). ACM, 1484–1506. https://doi.org/10.1145/3460120.3484539

Anjana Rajan, Lucy Qin, David Archer, Dan Boneh, Tancrede Lepoint, and Mayank Varia. 2018. Callisto: A cryptographic approach to detect serial predators of sexual misconduct. In *Proceedings of the 1st ACM SIGCAS Conference on Computing and Sustainable Societies (COMPASS)*.

Michael Rosenberg, Mary Maller, and Ian Miers. 2022. SNARKBlock: Federated Anonymous Blocklisting from Hidden Common Input Aggregate Proofs. In *2022 IEEE Symposium on Security and Privacy (SP)*. 948–965. https://doi.org/10.1109/SP46214.2022.9833656

Sarah Scheffler, Anunay Kulshrestha, and Jonathan Mayer. 2023. Public Verification for Private Hash Matching. In *2023 IEEE Symposium on Security and Privacy (SP)*. 253–273. https://doi.org/10.1109/SP46215.2023.10179349

Sarah Scheffler and Jonathan Mayer. 2023. SoK: Content Moderation for End-to-End Encryption. In *Proceedings of the Privacy Enhancing Technologies Symposium*. https://doi.org/10.56553/popets-2023-0060

Signal. 2013. Signal. https://signal.org/
Signal. 2023. Manage a Group. https://support.signal.org/hc/en-us/articles/360050427692-Manage-a-group
Signal-Desktop. 2018. Feature request: Edit sent messages. https://github.com/signalapp/Signal-Desktop/issues/2545
signalapp. 2023. Signal Release v6.33.0. https://github.com/signalapp/Signal-Desktop/releases/tag/v6.33.0
Slack. 2023a. Permissions by role in Slack. https://slack.com/help/articles/201314026-Permissions-by-role-in-Slack
Slack. 2023b. Slack is your productivity platform. https://slack.com/
Synapse. 2023a. Introduction. https://matrix-org.github.io/synapse/latest/
Synapse. 2023b. List Room API. https://matrix-org.github.io/synapse/latest/admin_api/rooms.html
The Draupnir Project. 2023. Draupnir. https://github.com/the-draupnir-project/Draupnir
Patrick P. Tsang, Man Ho Au, Apu Kapadia, and Sean W. Smith. 2010. BLAC: Revoking Repeatedly Misbehaving Anonymous Users without Relying on TTPs. *ACM Trans. Inf. Syst. Secur.* 13, 4 (2010), 39:1–39:33. https://doi.org/10.1145/1880022.1880033
Nirvan Tyagi, Paul Grubbs, Julia Len, Ian Miers, and Thomas Ristenpart. 2019a. Asymmetric Message Franking: Content Moderation for Metadata-Private End-to-End Encryption. In *Advances in Cryptology – CRYPTO 2019*, Alexandra Boldyreva and Daniele Micciancio (Eds.). Springer International Publishing, Cham, 222–250.
Nirvan Tyagi, Paul Grubbs, Julia Len, Ian Miers, and Thomas Ristenpart. 2019b. Asymmetric Message Franking: Content Moderation for Metadata-Private End-to-End Encryption. Cryptology ePrint Archive, Paper 2019/565. https://eprint.iacr.org/2019/565
Nirvan Tyagi, Julia Len, Ian Miers, and Thomas Ristenpart. 2022. Orca: Blocklisting in Sender-Anonymous Messaging. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, Boston, MA, 2299–2316. https://www.usenix.org/conference/usenixsecurity22/presentation/tyagi
Nirvan Tyagi, Ian Miers, and Thomas Ristenpart. 2019c. Traceback for End-to-End Encrypted Messaging. In *ACM CCS 2019: 26th Conference on Computer and Communications Security*, Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz (Eds.). ACM Press, 413–430. https://doi.org/10.1145/3319535.3354243
United Kingdom. 2021. Draft Online Safety Bill. https://www.gov.uk/government/publications/draft-online-safety-bill
US Department of Justice. 2020. International Statement: End-To-End Encryption and Public Safety. https://www.justice.gov/opa/pr/international-statement-end-end-encryption-and-public-safety
US Senate. 2023. S.1199: STOP CSAM Act of 2023. https://www.govtrack.us/congress/bills/118/s1199/text
Weijin Wang, Jingbin Liu, Yu Qin, and Dengguo Feng. 2017. Formal Analysis of a TTP-Free Blacklistable Anonymous Credentials System. In *Information and Communications Security - 19th International Conference, ICICS 2017, Beijing, China, December 6-8, 2017, Proceedings (Lecture Notes in Computer Science, Vol. 10631)*, Sihan Qing, Chris Mitchell, Liqun Chen, and Dongmei Liu (Eds.). Springer, 3–16. https://doi.org/10.1007/978-3-319-89500-0_1
WhatsApp. 2023. How to edit messages *(Help Center)*. https://faq.whatsapp.com/6614640168569481/?cms_platform=web
Li Xi and Dengguo Feng. 2014. FARB: Fast Anonymous Reputation-Based Blacklisting without TTPs. In *Proceedings of the 13th Workshop on Privacy in the Electronic Society, WPES 2014, Scottsdale, AZ, USA, November 3, 2014*, Gail-Joon Ahn and Anupam Datta (Eds.). ACM, 139–148. https://doi.org/10.1145/2665943.2665947
Gang Xu, Leonardo Aguilera, and Yong Guan. 2012. Accountable Anonymity: A Proxy Re-Encryption Based Anonymous Communication System. In *18th IEEE International Conference on Parallel and Distributed Systems, ICPADS 2012, Singapore, December 17-19, 2012*. IEEE Computer Society, 109–116. https://doi.org/10.1109/ICPADS.2012.25
Rupeng Yang, Man Ho Au, Junzuo Lai, Qiuliang Xu, and Zuoxia Yu. 2017. Lattice-Based Techniques for Accountable Anonymity: Composition of Abstract Stern's Protocols and Weak PRF with Efficient Protocols from LWR. Cryptology ePrint Archive, Report 2017/781. https://eprint.iacr.org/2017/781.
YouTube. 2023. Live stream on YouTube. https://support.google.com/youtube/topic/9257891?hl=en-GB&ref_topic=9257610
YouTube Help. 2023. How to use moderation tools. https://support.google.com/youtube/answer/10888907?hl=en&co=GENIE.Platform%3DDesktop
Kin Ying Yu, Tsz Hon Yuen, Sherman S. M. Chow, Siu Ming Yiu, and Lucas Chi Kwong Hui. 2012. PE(AR)2: Privacy-Enhanced Anonymous Authentication with Reputation and Revocation. In *ESORICS 2012: 17th European Symposium on Research in Computer Security (Lecture Notes in Computer Science, Vol. 7459)*, Sara Foresti, Moti Yung, and Fabio Martinelli (Eds.). Springer, Heidelberg, Germany, Pisa, Italy, 679–696. https://doi.org/10.1007/978-3-642-33167-1_39